

VestiCode 反思报告

本报告说明 VestiCode 的内部工作原理、关键设计决策、AI 工具使用情况，并对 Agent 核心循环 `AgentLoop.RunAsync` 做逐行解读。

配套：架构图见 [ARCHITECTURE.md](#)。

目录

1. 项目定位与设计目标
2. Agent 核心原理：ReAct 与四大组件
3. 核心循环 `AgentLoop.RunAsync` 逐行解读
4. 关键设计决策
5. `async/await` 的使用详解
6. 记忆策略设计
7. 错误处理与可观测性
8. AI 工具使用透明度
9. 问题诊断案例（真实 Bug 复盘）
10. 不足与改进方向

1. 项目定位与设计目标

VestiCode 是一个运行在终端中的**通用自主编码 Agent**：用户用自然语言描述编程任务，Agent 自行决定调用哪些工具、以什么顺序执行、何时停止，并在真实代码库上读写文件、执行命令、搜索代码、联网获取信息，直到任务完成。

设计围绕三个目标展开：

- **真实可用**：面向真实代码库工作，已可作为 `vesticode` 终端命令长期使用，而非一次性脚本。
- **推理透明**：每一步思考、工具调用、执行结果都实时呈现，用户始终能看到 Agent“在想什么、做了什么”。
- **结构清晰**：领域逻辑与界面彻底解耦，核心循环可被单元测试独立验证，也便于逐行讲解。

选择“编码 Agent”这一形态，是因为编码任务天然多步、强依赖“看到上一步结果再决定下一步”，能最充分地体现 ReAct 循环的价值——这正是 Agent 区别于一问一答式对话的根本所在。

2. Agent 核心原理：ReAct 与四大组件

ReAct = Reasoning + Acting，其循环为：

Thought（思考：分析现状，决定下一步）
→ Action（行动：调用一个工具/函数）
→ Observation（观察：拿到工具返回结果）
→ 重复，直到无需再调用工具 或 达到最大步数

四大组件分工明确：**LLM** 负责思考与决策，**Agent Loop** 把“决策→执行→观察”串成可迭代的闭环，**Tools** 是行动的手段，**Memory** 让多轮之间共享上下文。其中 **Agent Loop** 是关键——没有它，模型只能一问一答；有了它，模型才能“自我驱动”，把一个复杂目标拆成若干步逐一推进。

VestiCode 的 Agent Loop 即 `AgentLoop.RunAsync`，下面逐行解读其实现。

3. 核心循环 `AgentLoop.RunAsync` 逐行解读

文件：`src/VestiCode.Core/Agents/AgentLoop.cs`。这是整个系统的核心，以下按执行顺序逐段拆解。

3.1 方法签名

```
public async IAsyncEnumerable<AgentEvent> RunAsync(  
    ConversationHistory history,  
    [EnumeratorCancellation] CancellationToken cancellationToken = default)
```

- `async IAsyncEnumerable<AgentEvent>`：循环并非“算完再一次性返回”，而是边推理边 `yield` 事件（文本增量、工具调用、工具结果……）。调用方用 `await foreach` 实时消费，这是流式输出与“推理过程可观测”的根基。
- `ConversationHistory history`：既是输入也是被持续改写的对象——工具结果会回填进去，构成下一轮的上下文。
- `[EnumeratorCancellation]`：把 `await foreach` 传入的取消令牌正确绑定到这个异步迭代器，使外部 `Esc / Ctrl+C` 能中断循环。

3.2 外层轮次循环

```
for (var round = 1; round <= _maxRounds; round++)  
{  
    if (cancellationToken.IsCancellationRequested)  
    {  
        yield return new AgentDoneEvent(AgentDoneReason.Cancelled);  
        yield break;  
    }  
}
```

- `_maxRounds` 是安全护栏，防止模型陷入无限的工具调用循环。
- 每轮开头先检查取消：在轮次边界取消属于“优雅停止”——产出 `Done(Cancelled)` 后 `yield break` 干净退出，而不是抛异常打断。

3.3 进入 LLM 前的上下文压缩

```
if (_compressor is not null)  
{  
    var compression = await _compressor.CheckAndCompressAsync(history,  
        cancellationToken)...
```

```

        if (compression.WarningIssued)
            yield return new ContextWarningEvent(compression.EstimatedTokens,
            _compressor.ContextWindow);
        if (compression.WasCompressed)
            yield return new CompactionEvent(compression.MessagesCompressed,
            compression.EstimatedTokensSaved);
    }

```

- 调用 LLM 前先检查 token 是否逼近上下文窗口：达 70% 仅发警告事件，达 90% 才真正用 LLM 生成结构化摘要替换早期消息。
- `_compressor` 为可空依赖：子 Agent / 测试等场景可以不注入压缩器。这是贯穿 `AgentLoop` 的“可选依赖”模式——非核心能力一律以可空构造参数注入，缺省即关闭。

3.4 组装消息与工具定义

```

yield return new RoundStartEvent(round, _maxRounds);
// Hook: ROUND_START ...
var messages = AssembleMessages(history, round);
var toolDefs = BuildToolDefs();

```

- `AssembleMessages` 拼装本轮发送给模型的完整消息：动态生成的 **System Prompt**（含实时环境信息）+ 调度模式工作流（若激活）+ 已激活 **Skill** 的 **SOP** + 历史消息 + 每轮注入（如 **plan-only** 提醒），最后再过一层“超大工具结果截断”。
- `BuildToolDefs` 把已注册工具导出为 LLM 可理解的 Function Calling 定义；若存在 **Skill** 工具白名单或调度模式，会相应过滤可见工具集。

3.5 调用 LLM 并流式累积

```

await foreach (var item in _provider.ChatStreamAsync(messages, toolDefs,
cancellationToken))
{
    switch (item)
    {
        case TextDelta td:      textBuffer.Append(td.Text); yield return
new TextDeltaEvent(td.Text); break;
        case ThinkingDelta th:  yield return new ThinkingEvent(th.Text,
th.Label); break;
        case ToolCallReady tc:  toolCalls.Add(tc.Call); yield return new
ToolCallEvent(tc.Call); break;
        case UsageReport ur:    yield return new UsageEvent(ur.InputTokens,
ur.OutputTokens); break;
        case StreamError se:    ...; yield return new ErrorEvent(...);
        hadError = true; break;
    }
    if (hadError) yield break;
}

```

这是 **Thought + Action** 的产生过程：

- 模型一边输出文本（思考/回答），一边可能产出工具调用请求；两者交错到达。
- 每个增量立即 **yield** 给上层，于是用户看到逐字流式输出。
- 文本累积进 **textBuffer**，工具调用累积进 **toolCalls**，供本轮后续判断。
- 若 LLM 报错（如 401、限流）：产出 **ErrorEvent** 并 **yield break**，**进程不崩溃**，用户可修改配置后重试。

3.6 终止判断：无工具调用即完成

```
if (toolCalls.Count == 0)
{
    if (textBuffer.Length > 0)
        history.AddAssistantMessage(textBuffer.ToString());
    yield return new AgentDoneEvent(AgentDoneReason.NoToolCall);
    yield break;
}
```

- 模型不再请求工具，意味着任务完成。把最终回复写入历史，产出 **Done(NoToolCall)**，结束循环。

3.7 记录本轮 assistant 的工具调用

```
history.AddRawMessage(ChatMessage.FromToolCalls(toolCalls,
textBuffer.ToString()));
```

- 把“模型本轮说了什么 + 请求了哪些工具”作为一条 assistant 消息写入历史。**必须先写 assistant 的 tool_use**，再写对应的 **tool** 结果，否则二者不配对，下一轮请求会被 LLM API 拒绝。

3.8 分批执行：读类并发 / 写类串行

```
var (reads, writes) = PartitionByCategory(toolCalls);
```

按工具 **Category** 把本轮调用分成只读组与写入组。

读类——先逐个安全门控，通过的再并发执行：

```
foreach (var call in reads)
{
    var gate = GateTool(call);
    if (gate.Decision == SecurityDecision.Ask) { ...弹 HITL，等用户裁决... }
    else if (gate.Decision == SecurityDecision.Deny) { yield return
BlockTool(...); continue; }
    allowedReads.Add(call);
}
if (allowedReads.Count > 0)
```

```

{
    var results = await ExecuteConcurrentAsync(allowedReads,
cancellationTokens); // Task.WhenAll
    for (var i = 0; i < allowedReads.Count; i++) {
        yield return new ToolResultEvent(allowedReads[i].Name,
allowedReads[i].Arguments, results[i]);
        AppendToolResult(history, allowedReads[i], results[i]); //
Observation 回填
    }
}

```

- 只读操作互不影响，因此用 `Task.WhenAll` 并发执行以提速。
- 每个结果都通过 `AppendToolResult` 回填历史（Observation），模型下一轮才能看到执行结果。

写类——逐个门控并串行执行：

```

foreach (var call in writes)
{
    if (cancellationToken.IsCancellationRequested) { yield return
Done(Cancelled); yield break; }
    if (!_planOnly && !PlanModeAllowed.Contains(call.Name)) { yield return
BlockTool(... "plan-only..."); continue; }

    var gate = GateTool(call); // Ask → 弹 HITL; Deny → 拦截并把原因
回灌
    ...
    if (_hookEngine...) { var reject = await Hook(TOOL_PRE_EXEC); if (reject)
{ BlockTool; continue; } }

    var result = await ExecuteSingleAsync(call, cancellationToken);
    yield return new ToolResultEvent(call.Name, call.Arguments, result);
    AppendToolResult(history, call, result);
    if (_hookEngine...) await Hook(TOOL_POST_EXEC);
}

```

- 写操作有副作用，必须串行，避免两个 edit/write 竞争同一文件。
- 写前要过三道关卡：plan-only 拦截 → 安全门控（可能弹 HITL）→ Hook 拦截。
- HITL 的精妙处：`yield return HitlRequestEvent(... tcs)` 把请求连同同一个 `TaskCompletionSource` 抛给界面，随后 `await tcs.Task` 挂起等待用户按键——这就是“人在回路”，且等待期间不占用线程（详见第 5 节）。

3.9 兜底：达到最大轮次

```

} // for
yield return new AgentDoneEvent(AgentDoneReason.MaxRounds);
}

```

- 若跑满 `MaxRounds` 仍未完成，产出 `Done(MaxRounds)`，给出明确的终止原因。

一句话概括：组装消息 → 流式调用 LLM → 没有工具调用就结束；有就分批执行（读并发 / 写串行，每步先过安全门控、再把结果回填历史）→ 进入下一轮，直到完成或触顶。

4. 关键设计决策

决策	做法	理由
LLM 接入	HTTP 直连 + 自研 SSE 流式解析，统一为 <code>ILlmProvider</code> 抽象	完全掌控 ReAct 循环、流式增量解析与多协议适配（OpenAI / Anthropic 两套线缆格式）；不依赖框架黑盒，便于对每行代码负责。
循环返回类型	<code>async IEnumerable<AgentEvent></code>	天然支持流式与可观测：边推理边产出事件供界面实时渲染；并能用 <code>yield break</code> 在任意点干净终止。
界面解耦	事件驱动——循环只产出事件，界面负责消费	领域逻辑零 UI 依赖，可替换控制台 / Web / 桌面前端，且核心循环能被单元测试独立驱动。
工具并发模型	读类并发、写类串行	只读操作无副作用可并行提速；写操作有副作用必须串行以防竞态。
人在回路	<code>TaskCompletionSource</code> 异步握手	循环 <code>yield</code> 出授权请求并 <code>await</code> TCS，把“等用户按键”变成不阻塞线程的异步等待。
记忆压缩	两段式：70% 警告 / 90% 压缩 + 熔断	避免过早压缩损失上下文；摘要连续失败则熔断，防止反复消耗 token。
Provider 配置	最小配置（Name/Model/ApiKey）+ <code>providers.json</code> 推导协议与基址	用户配置极简，新增后端只需追加一行映射、无需改代码。
多 Agent 隔离	git worktree 物理隔离 + 分支合并	多成员并发改文件互不干扰，再用 git merge 汇总，天然解决并行与冲突。
安全模型	黑名单 + 沙箱 + 三档 + HITL 多层纵深	危险命令直接拦、路径越界拦、其余按档位询问，分层防御而非单点。
可选依赖	非核心能力以可空构造参数注入	同一 <code>AgentLoop</code> 既可“全功能”运行，也可在测试 / 子 Agent 中以最小依赖运行。

5. async/await 的使用详解

异步与并发是本项目的核心机制。

(1) 异步流（贯穿全局）

```
async IEnumerable<AgentEvent> RunAsync(...)           // Agent 事件流
IEnumerable<LlmStreamItem> ChatStreamAsync(...)       // LLM SSE 流
```

```
await foreach (var item in _provider.ChatStreamAsync(...))
```

LLM 响应是逐块到达的网络流，用 `IAsyncEnumerable` + `await foreach` 即可“收到一块、渲染一块”，实现真正的流式输出，而非等整段返回。

(2) I/O 密集 → 等待时释放线程 LLM 调用、文件读写、shell 执行、HTTP 抓取都是 I/O 等待型操作。`await` 在等待期间把线程归还线程池，CLI 不会卡死，仍能即时响应 `Esc / Ctrl+C`。

(3) 并发执行：Task.WhenAll

```
var tasks = calls.Select(c => ExecuteSingleAsync(c, ct)).ToArray();
return await Task.WhenAll(tasks); // 同一轮多个只读工具并发
```

(4) 异步握手：TaskCompletionSource

```
var tcs = new HitlSource(); // 以
RunContinuationsAsynchronously 创建
yield return new HitlRequestEvent(..., tcs);
var verdict = await tcs.Task; // 挂起等待用户按键，不占线程
```

界面读到按键后调用 `tcs.SetResult(决定)`，`await` 处随即恢复。

`RunContinuationsAsynchronously` 确保设置结果时不会同步回到 UI 线程造成重入。

(5) 串行化：SemaphoreSlim PersistentShell 用信号量保证命令串行（shell 本质串行），且 `await _gate.WaitAsync()` 不阻塞线程。

(6) ConfigureAwait(false) 领域层所有 `await` 均加 `ConfigureAwait(false)`，不强制回到原同步上下文，减少上下文切换并规避潜在死锁。

6. 记忆策略设计

- **短期记忆：ConversationHistory** 保存当前会话的全部消息（System / User / Assistant / Tool 四种角色）；工具结果作为 `tool` 消息回填，构成“工作记忆”。
- **持久记忆：JsonlSessionStore** 把会话写为 `{id}.jsonl`（逐行消息，追加写 $O(1)$ ）+ `{id}.meta.json`（标题/时间/模型等）。加载时具备容错：跳过损坏行、在未配对的 `tool_use` 处截断、距上次活跃超 30 分钟注入时间提醒；`--resume` 可续接上次会话。
- **上下文压缩**：逼近窗口时用 LLM 生成结构化摘要替换早期消息，保留最近 4 条原文，兼顾“记得久”与“记得清”。
- **长期偏好：AutoNoteManager** 每 5 轮按四类（用户偏好 / 纠正反馈 / 项目知识 / 参考资料）增量提炼笔记，跨会话可用；每类只提炼本类信息，避免相互污染。

7. 错误处理与可观测性

- **LLM 错误**：流中出现 `StreamError` → 转为 `ErrorEvent` 红色提示，进程不崩溃。

- **工具错误**: `ToolExecutor` 统一捕获异常与超时 → `ToolResult.Fail(原因)`，原因回灌模型，促其自我修正。
- **安全 / Hook 拦截**: 被拒原因写回历史，模型据此换一种做法（如改用相对路径）。
- **可观测性**: 每个 token、思考、工具调用、工具结果、上下文压缩、HITL 请求都以事件实时呈现，推理过程对用户完全透明；后台同时有 Serilog 滚动文件日志可供排查。

8. AI 工具使用透明度

如实记录本项目的 AI 使用情况：

- **开发方式**: 本项目在 AI 辅助下开发，AI 协助产出了样板代码、Provider 的 SSE 解析初稿、TUI 渲染细节等。
- **我主导与把控的部分**: 整体分层与模块边界；ReAct 循环中“读并发 / 写串行 + 安全门控 + HITL”的编排；两段式记忆压缩策略；多 Agent 的 git worktree 隔离方案；Provider 最小配置 + catalog 推导的设计；以及对每一处 AI 产出的审阅、调试与修正。
- **典型的人工修正**:
 - 工具结果显示串台 → 让 `ToolResultEvent` 携带本次 `Arguments`，按调用而非工具名定位；
 - `ls && rm -rf /` 绕过黑名单 → 改为对复合命令按分隔符逐段检查；
 - 后台清理器误删刚创建的 worktree → 让“过期”判定真正按闲置时长生效。

9. 问题诊断案例（真实 Bug 复盘）

以下均为开发过程中真实定位并修复的缺陷，体现对系统行为的实际掌控。

现象	根因	修复
读到的文件内容比 edit 看到的 的多了一个不可见字符	新建文件带 UTF-8 BOM，read 未 去除	<code>ReadFileTool</code> 去掉首个 U+FEFF
同一轮多次调用同一工具，结 果显示的文件名串台	用工具名作键，后者覆盖前者	<code>ToolResultEvent</code> 改为携带本次 <code>Arguments</code>
<code>ls && rm -rf /</code> 未被黑名 单拦下	只检查命令首个 token	按 <code>&& \ \ ; \ </code> 等分隔符逐段 检查前缀
macOS 上替换二进制后 <code>zsh: killed</code>	原地覆盖已签名二进制导致内核签 名页失效 (SIGKILL)	安装脚本先 rm 旧 inode 再 cp，并 ad-hoc 重签名

10. 不足与改进方向

- **未实现 RAG / 向量检索**: 长期知识目前依赖笔记文件，尚未接入向量库做检索增强。
- **MCP 仅客户端**: 能连接外部 MCP server，但未实现 MCP 服务端。
- **团队拆解依赖模型质量**: 复杂目标的子任务划分效果取决于 LLM；合并冲突虽有 LLM 裁决兜底，极端情况下仍可能失败回滚。
- **界面仅控制台**: 得益于事件驱动解耦，未来可平滑扩展 Web / 桌面前端。